

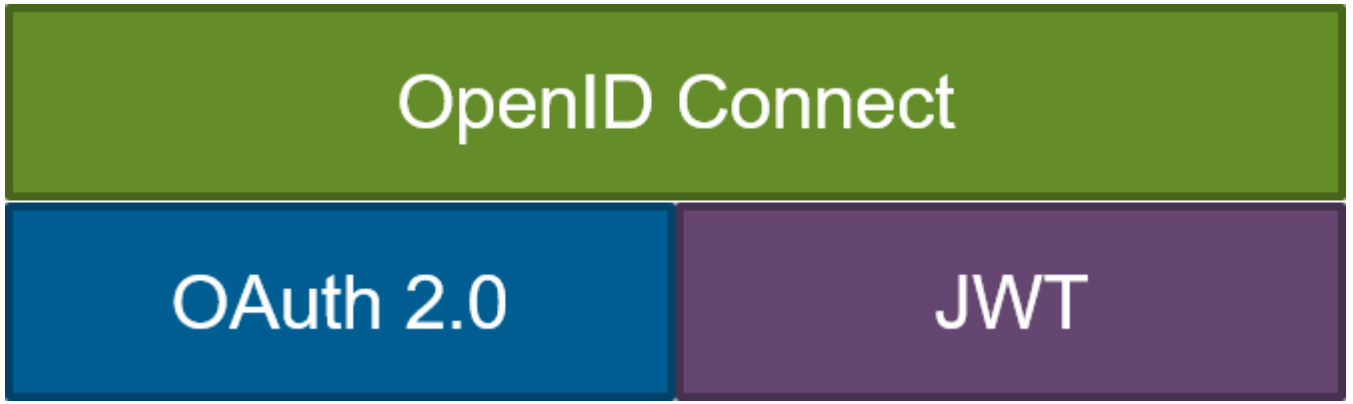
IAM

Authentifizierungsprotokolle - OIDC

Prof. Dr. Annett Laube, HS 2025, HSLU

OpenId Connect - OIDC

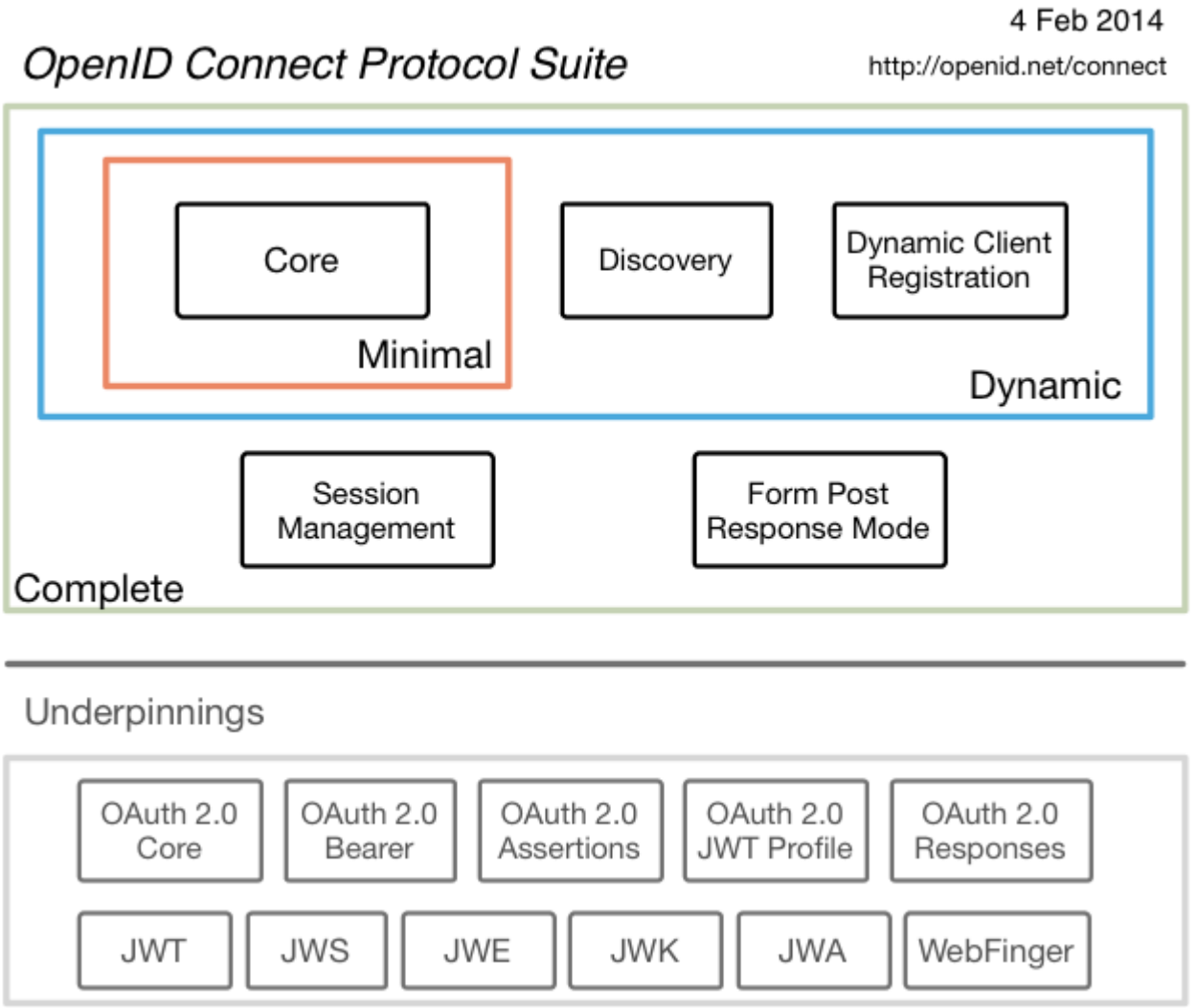
- **ist eine Authentifizierungs- und Identitätsschicht oberhalb von OAuth 2.0**.
- OpenID Connect 1.0 (OIDC) ermöglicht Clients,
  - **Informationen** über den **Benutzer** zu erhalten,
  - sowie dessen **Identität zu überprüfen,**
  - und damit die Einrichtung von Anmeldesitzungen.
  - Sämtliche Informationen werden als JSON-Objekte (JWT) ausgetauscht.



OIDC Protocol Suite

Enthält neben dem Protokoll u.a.:

- Discovery – Definiert, wie OpenID Providers dynamisch verwendet werden können
- Dynamic Registration – Definiert die dynamische Registrierung von Clients
- Session Management – Definiert Session Management, inkl. Logout

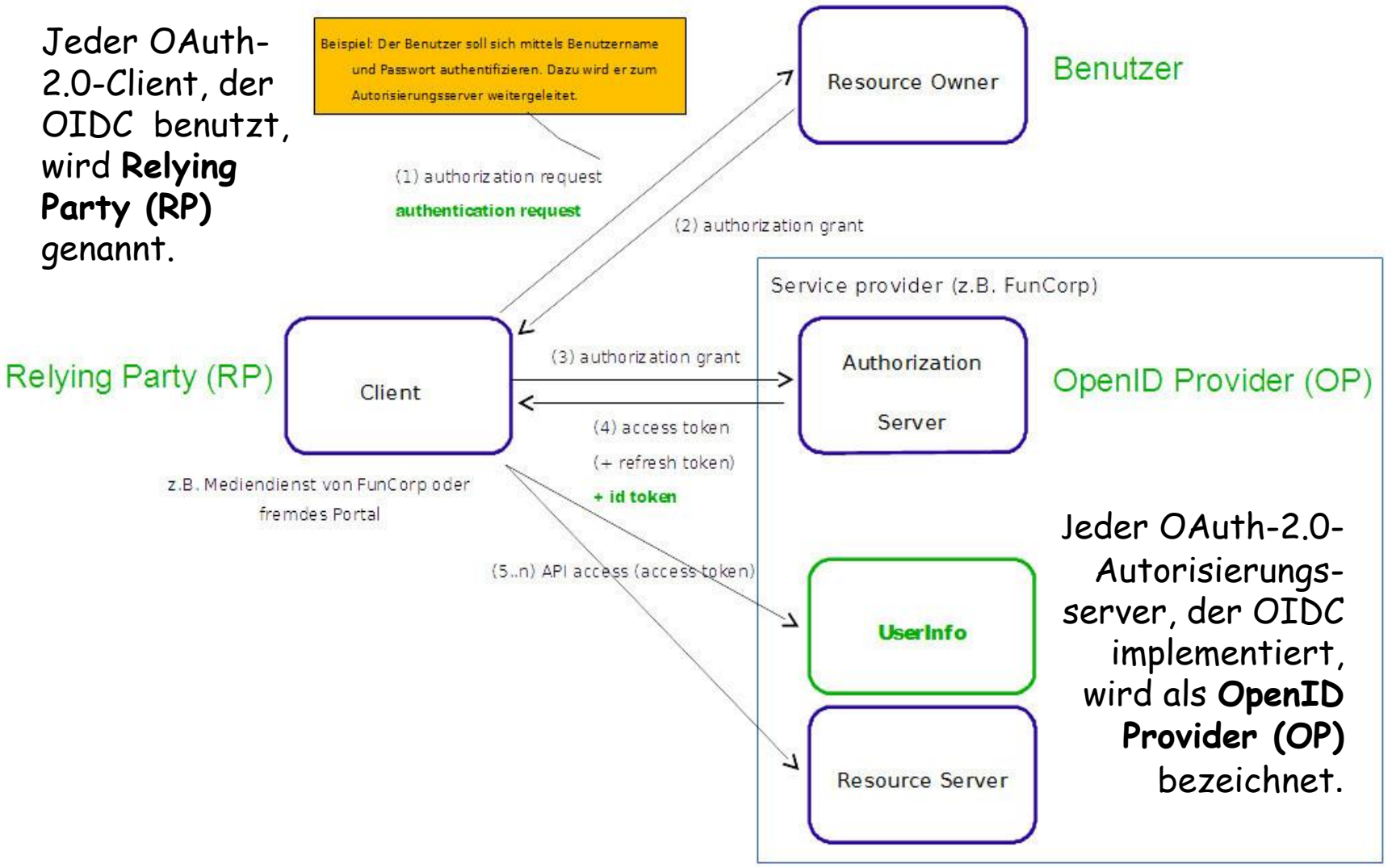


OIDC - Protokollfluss

Der oAuth-Protokollfluss wurde um die grünen Teile erweitert.

Durch die Angabe von *scope = openid* wird die OpenID-Erweiterung aktiviert.

Als Ergebnis wird dann zusätzl. das *ID Token* an den Client zurückgegeben.



userInfo Endpoint

- Eine oAuth-geschützte **API** zum Zugriff auf Benutzerinformationen unter Verwendung von Access Tokens, den **UserInfo Endpunkt**.
- Diese wird vom *Authorization Server* bereitgestellt und liefert Informationen zum jeweiligen Benutzer, auf die der Client zugreifen darf.
- Das geschieht über Backchannel.

Weitere Endpoints

- **Authorization Endpoint**:
  - Hier wird der Benutzer autorisiert und sein Einverständnis abgeholt.

- **Token Endpoint**:
  - Hier schickt der Client den Authorization Code, um sie gegen *ID Token* und *Access Token* (optional Refresh Token einzutauschen)
    - `grant-type = authorization_code`
  - Wird auch genutzt, um ein Refresh Token gegen eine neues Access Token einzutauschen.
    - `grant-type = refresh_token`

- **Logout Endpoint**: Beenden der Benutzer-Session (initiiert ein Logout am OP).
- …

Protokollvarianten zur Authentifizierung (1/2)

Wie bei oAuth 2.0 gibt es verschiedene Möglichkeiten, wie der Client mit dem OIDC Provider interagieren kann:

- **Authorization Code Flow**:
  - Der Authorization Code wird gegen ein ID Token und Access Token (optional Refresh Token) am Token Endpunkt eingetauscht, dabei ist meist eine Client-Authentifizierung erforderlich
  - ermöglicht einen langlebigen Zugriff (durch die Verwendung von Refresh-Token)
  - für Web-Clients, wobei die Tokens immer direkt über einen gesicherten Kanal an das Backend per Back-Channel geschickt werden
  - Für Native Apps (Android oder iOS): es wird speziell ein OIDC Single-Sign-On verwendet (Erweiterung zum Authorization Code Flow)
  - bekannte OpenID Connect Provider sind Apple, Google und Microsoft (Nicht Facebook/Meta)

Protokollvarianten zur Authentifizierung (2/2)

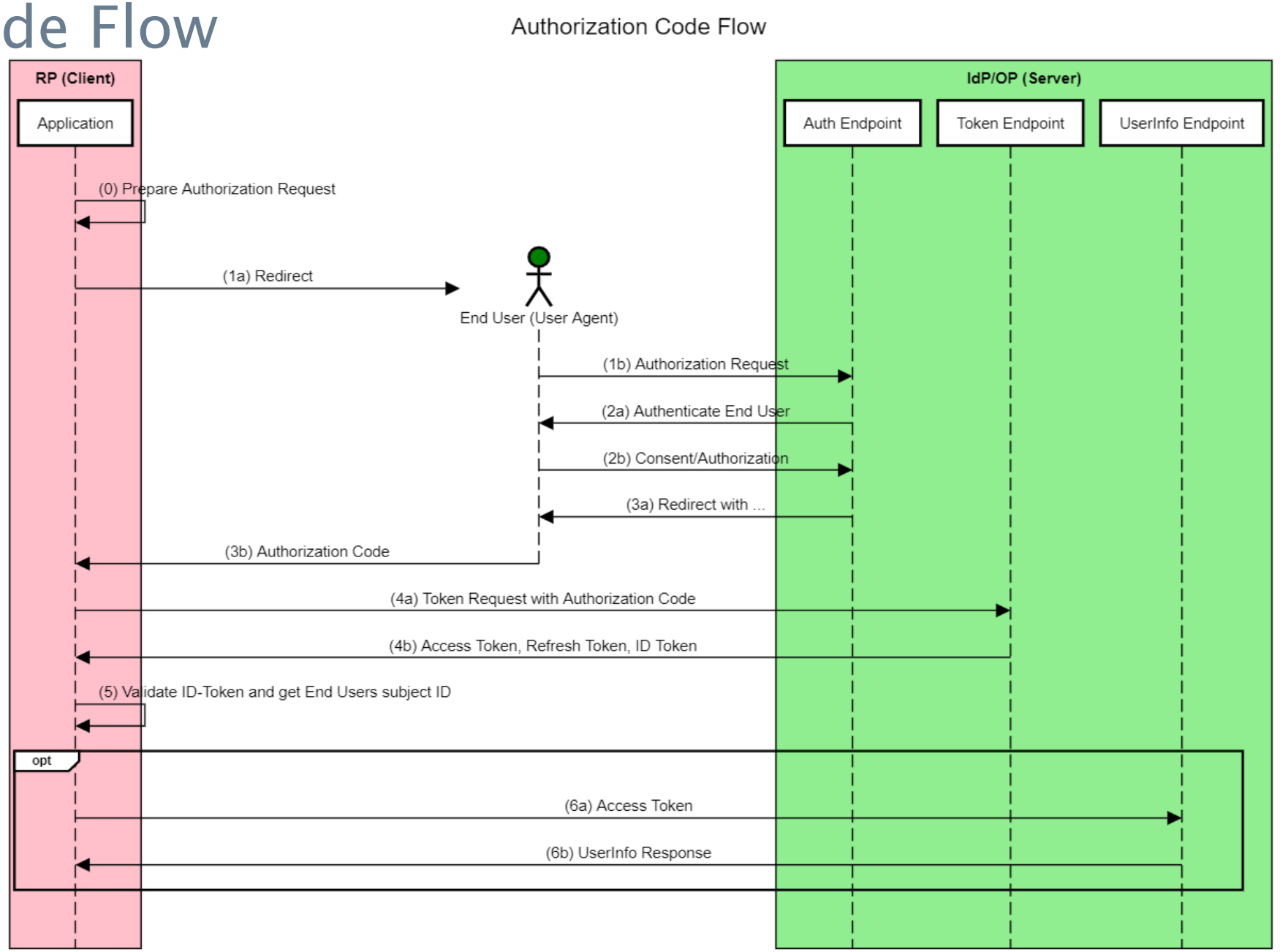
- **Implicit Flow (not recommended)** :
  - Am einfachsten zu implementieren (nur ein Request an OP)
  - Keine Client-Authentifizierung erforderlich, nur redirect URI wird überprüft
  - Ursprünglich für Anwendungen, die nicht in der Lage sind, Geheimnisse sicher zu speichern, z.B. Clients, die in einem Browser unter Verwendung einer Skriptsprache implementiert sind

- **Hybrid Flow**:
  - Kombination aus Authorization Code und Implicit Flow
  - ebenfalls für Mobile- und Web-Clients, wobei die ID-Tokens direkt die Clients geschickt werden, und das Backend per Back-Channel alle Tokens erhält und verwaltet

Authorization Code Flow

Der Ablauf generell wie bei *oAuth 2.0. Authorization Code Flow*:

- Zusammen mit dem Access Token (4b) wird auch das *ID Token* mitgeliefert.



Authentication Request

Die fett gedruckten Parameter sind obligatorisch, die kursiven optional:

- **reponse_type:** Angabe zum gewünschten Flow (z.B. *code* für Access & ID Token)
- **scope**: Angabe, welche Benutzerdaten angefordert werden (z.B. `scope=openid%20profile%20email`)
- **client_id**: Identifier des Clients (RP)
- **redirect_uri:** Rücksende-Adresse für die Authentisierungsbestätigung des Users
- **state:** String, um request und callback zuzuordnen
- *nonce*: Ein String, um die Client Session dem ID Token zuzuordnen. Dies verhindert oder erschwert Replay-Angriffe.
- *max_age*: Maximale Zeitdauer einer User Session, also nach welcher Zeit sich ein User neu beim OpenID Connect Provider authentifizieren muss. Die Zeitdauer wird in Sekunden angegeben.

```
https://YOUR_DOMAIN/authorize?
  response_type=code&
  client_id=YOUR_CLIENT_ID&
  redirect_uri=https://YOUR_APP/callback&
  scope=openid%20profile%20email&
  state=YOUR_STATE_VALUE
```

ID Token

- enthält Identitätsdaten und Informationen über den Authentifizierungsprozess in Form eines sogenannten JSON Web Token (JWT).
- Um dieses gegen Modifikation und andere Angriffe abzusichern, lässt es sich digital signieren und verschlüsseln

- **Aufbau*:*** Header, Payload, Signatur

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ
zdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4
gRG9lIiwiaWF0IjoxNTE2MjM5MDIyfQ.SflKxwRJ
SMeKKF2QT4fwpMeJf36POk6yJV_adQssw5c
```

- **Codierung**

jwt = base64UrlEncode(header) + "." + base64UrlEncode(payload) + "." + base64UrlEncode(hash)

ID Token als JWT

- Header: Info zu Signatur und Verschlüsselung

```
{
   "alg":  "HS256",
   "typ":  "JWT"
}
```

- Payload: enthält einen sogenannten *Claims* zum Benutzer oder zum Authentifizierungsprozess.

```
{
  "iss": "https://accounts.funcorp.com",
  "sub": "786767677",
  "aud": "SDFGHJKLUZTREDFGHJ",
  "exp": 1388700360000,
  "iat": 1388696760000,
  "auth_time": 1388696760000
  "acr": "urn:funcorp:com:level-1"
}
```

iss: Issuer = OP

sub: SubjectID = BenutzerID auf der RP

aud: Audience (Empfänger) = ClientID der RP

"iat" (Issued At) und "exp" (Expires At) = Gültigkeit des Token

"auth_time" = Zeitpunkt der Authentifizierung

"acr" = Qualität/Policy der Authentifizierung“

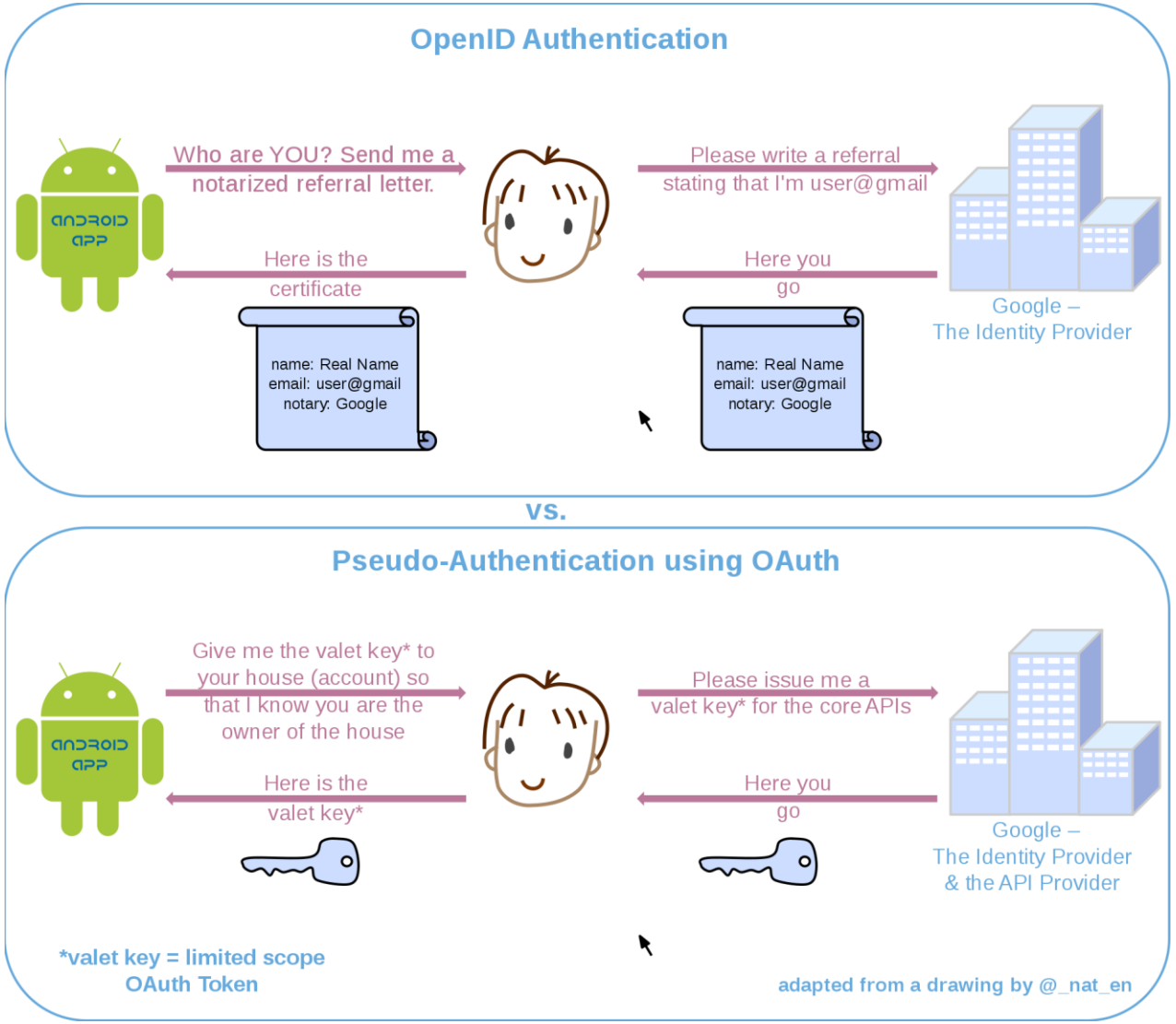
Vorteile von OIDC

- **Benutzerfreundlichkeit**: Zentrales Login beim OpenID Provider: Benutzer müssen kein zusätzliches Login erstellen, sondern können sich mit gängigen Logins anmelden. Das spart Zeit und Aufwand für die Benutzer.
- **Entlastet Anbieter (RPs):** RPs können Authentifizierung und Benutzerverwaltung an den OpenID Provider delegieren. Die Entwickler können sich somit auf die Hauptfunktionen von Applikationen konzentrieren
- **Sicherheit**: Alle übermittelten Daten und Parameter müssen mit TLS verschlüsselt werden. Alles Daten zwischen Client Applikation (RP) und OpenID Connect Provider über einen gesicherten Backchannel ausgetauscht (TLS-Handshake).
- **Native SSO**: Erweiterung für Single-Sign-On für Native Apps verfügbar.
- **Popularität**: ausgereifte und weit verbreitete Protokolle (Social Logins).

Nachteile von OIDC

- **Bedingte Anonymität**: Wenn sich ein User für eine Applikation bei einem der bekannten OpenID Connect Provider authentifiziert, ist er anschliessend mit dem jeweiligen Kontonamen angemeldet. Verwendet der User bei diesem Profil seinen echten Namen, ist dieser dann in der Applikation ersichtlich.
- **Phishing**: User können nach wie vor auf gefälschte Login-Seiten stossen und dort unwissentlich ihre Logindaten an Angreifer übermitteln.
- **Kehrseite der Benutzerfreundlichkeit (Abhängigkeit):** Wenn Benutzer ein einziges Login für verschiedenste Anwendungen nutzt und dieses Login kompromittiert wird, kann dies Folgen für alle Applikationen haben, für welche der User dieses Login verwendet hat.
- **Desinteresse der Entwickler**: OIDC hat relativ komplexe Protokolle, die nicht einfach zu verstehen und korrekt zu implementieren sind.

Vergleich OIDC mit oAuth



Problem der Pseudo-Authentifizierung:

Der «valet key» (oAuth access token) ist nicht personalisiert, d.h. jeder der ihn vorweisen kann, bekommt Zugriff.

Diskussion

1. Was für Token gibt es bei oAuth2.0 und OIDC?
   - In welchen Protokollflüssen werden sie verwendet?
   - Was beinhalten die Tokens?
   - Wie werden sie übertragen?
2. Nennen Sie Unterschiede von oAuth 2.0 und OIDC?
3. Welche Flows gibt es in oAuth2.0/OIDC? Wann werden Sie eingesetzt?

Quellen

- https://openid.net/connect/
- https://openid.net/specs/openid-connect-core-1_0.html
- https://www.heise.de/developer/artikel/OpenID-Connect-Login-mit-OAuth-Teil-1-Grundlagen-2218446.html
- https://backstage.forgerock.com/docs/am/5/oidc1-guide/
- https://www.oose.de/blogpost/oauth-openid-connect-und-jwt-wie-haengt-das-alles-zusammen-teil-2/